SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

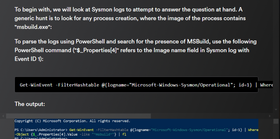
Task 1. Has msbuild executed malware on the machine?

To begin with, we will look at Sysmon logs to attempt to answer the question at hand. A generic hunt is to look for any process creation, where the image of the process contains "msbuild.exe":

To parse the logs using PowerShell and search for the presence of MSBuild, use the following PowerShell command ("$_.Properties[4]" refers to the Image name field in Sysmon log with Event ID 1):

```
Get-WinEvent -filterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=1} | where
```

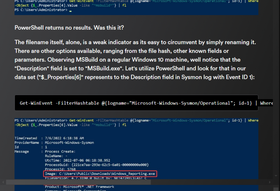
The output:



PowerShell returns no results. Was this it?

The filename itself, alone, is a weak indicator as its easy to circumvent by simply renaming it. There are other options available, ranging from the file hash, other known fields or parameters. Observing MSBuild on a regular Windows 10 machine, will notice that the "Description" field is set to "MSBuild.exe". Let's utilize PowerShell and look for that in our data set ("$_.Properties[6]" represents to the Description field in Sysmon log with Event ID 1:

```
Get-WinEvent -filterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=1} | where
```



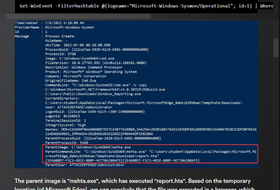
Note that the binary image is "C:\Users\Public\Downloads\Windows_Reporting.exe" which starts with a command line parameter of "Windows_Reporting.exe".

Task 2. What was the initial stager?

Search for the presence of CMD (Command Prompt), use the following PowerShell command ("$_.Properties[4]" refers to the Image name field in Sysmon log with Event ID 1): now focusing on the ParentImage and ParentCommandLine parameters:

```
Get-WinEvent -filterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=1} | where
```

The parent image is "mshta.exe", which has executed "report.hta". Based on the temporary location (of Microsoft Edge), we can conclude that the file was executed in a browser, which then started Windows_Reporting.exe.



renamed to Windows_Reporting.exe and located in an odd location.

Task 3. How was the malware downloaded? Why did the attacker choose this method?

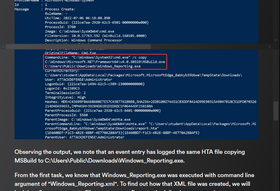
Let's begin tracing backwards - so far we know that MSBuild is present in an odd location under different name, and that an HTA file was used to execute the file. Since we know the file is called "Windows_Reporting.exe", let's look at all process creation events that have referenced this file with the following command ("$_.Properties[10]" represents to the CommandLine field in Sysmon log with Event ID 1):

```
Get-WinEvent -filterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=1} | where
```

Observing the output, we note that an event entry has logged the same HTA file copying MSBuild to C:\Users\Public\Downloads\Windows_Reporting.exe.

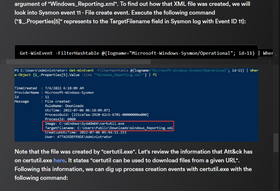
From the first task, we know that Windows_Reporting.exe was executed with command line argument of "Windows_Reporting.xml". To find out how that XML file was created, we will look into Sysmon event 11 - File create event. Execute the following command ("$_.Properties[5]" represents to the TargetFilename field in Sysmon log with Event ID 11):

```
Get-WinEvent -filterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=11} | wher
```



Note that the file was created by "certutil.exe". Let's review the information that ATT&CK has on certutil.exe [here](). It states "certutil can be used to download files from a given URL". Following this information, we can dig up process creation events with certutil.exe with the following command:

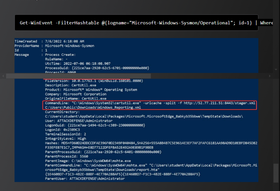
```
Get-WinEvent -filterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=1} | where
```



It appears that certutil.exe was utilized to download the XML file from http://52.77.211.0:4443.

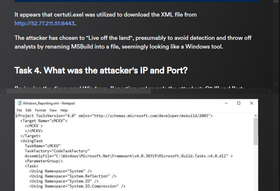
The attacker has chosen to "Live off the land", presumably to avoid detection and throw off analysts by renaming MSBuild into a file, seemingly looking like a Windows tool.

Task 4. What was the attacker's IP and Port?

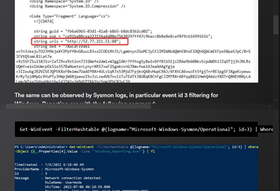


The same can be observed by Sysmon logs, in particular event id 3 filtering for

```
Get-WinEvent -filterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=3} | where
```



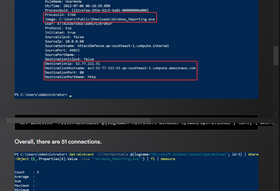
Overall, there are 51 connections.

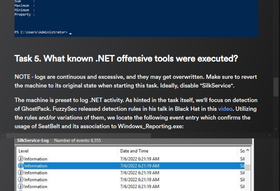


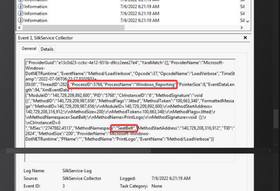
Task 5. What known .NET offensive tools were executed?

NOTE - logs are continuous and excessive, and they may get overwritten. Make sure to revert the machine to its original state when starting this task. Ideally, disable "SilkService".

The machine is preset to log .NET activity. As hinted in the task itself, we'll focus on detection of GhostPack. FuzzySec released detection rules in his talk in Black Hat in this video. Utilizing the rules and/or variations of them, we locate the following event entry which confirms the usage of SeatBelt and its association to Windows_Reporting.exe:







KEY TAKEWAYS

1. Robust detection rules as well as alternative/additional data sources are required to perform high quality hunts
2. Adversaries often choose to hide in plain sight